

COMP1531 · MAJOR PROJECT

Unigotchi — Iteration 1

Basic functionality & tests · Ship as a team

```
$ npm test # the stubs have to actually work now
```

From design to working, tested code

Your stubs need to work now, and you prove it with tests.

ITERATION 0 (done)

- Understand the requirements
- Design the function interfaces
- Set up how the team works
- Stub functions - shape only, no logic

ITERATION 1 (now)

- Follow good git, & teamwork practice
- Write tests for + implement the prescribed core functions
- Implement your designed functions from Iter 0

NO web server yet — plain JS through the interface

What's new in the starter code

TESTING TOOLCHAIN · package.json

"type": "module" → ESM project
use import / export – NOT require

```
vitest + @vitest/coverage-v8  
npm test           # vitest run  
npm run coverage   # + coverage
```

WORKED EXAMPLE + WHERE CODE GOES

echo.js + echo.test.js - a function & its test

```
import { describe, expect, test } from 'vitest'
```

dataStore.js - getData(), loadData(path), data={}

Fill the empty files:

```
auth.js · simulation.js · other.js (+ designed)
```

First word of the function name → its file

The example universe in the dataStore.js comment is illustrative and LOOSER than the real schema
The SPEC is the source of truth - validate against section 3.3, not the comment.

Mechanics you implement (1): validate + stat maths

UNIVERSE VALIDATION → INVALID_UNIVERSE

Only assumption: universeData is an object.

Validate everything else:

- courses: 1–6 entries
 - courseId: 4 UPPERCASE letters + 4 digits
 - name: 2–40 characters
- exactly 3 exams per course
 - examId 2–20 · courseId matches · name 2–40
 - requiredStudyHours: positive integer

Don't hard-code — automarking uses unseen universes

STAT FORMULAS & DURATION

$\Delta \text{energy} = \text{energy/hr} \times \text{hours}$ $\Delta \text{happiness} = \text{happiness/hr} \times \text{hours}$

Stats are integers, clamped to 0–100 after every update

duration = whole hours, min 1 → else INVALID_DURATION

Activity	Energy/hr	Happy/hr	Study/hr
Doom Scrolling	+3	+2	0
Studying	−15	−5	+1
Sleeping	+15	+2	0
Eating	+10	+3	0
Exercising	−10	+6	0

Study 1h then sleep 2h: energy 100→85→100 (clamped), happy 50→45→49

Mechanics you implement (2): constraints, exams, status

ACTIVITY CONSTRAINTS

Studying: max 3 consecutive hrs, then a different activity ≥ 1 hr before studying again

Exercising: must have eaten in the last 4 hrs + 1 hr cooldown from previous exercise end

Windows measured from when an activity ENDS, not starts

Constraints also cap an action's max duration

EXAMS & COURSE STATUS

PASS if `studyHoursCompleted` $\geq$ `requiredStudyHours`

Study hours tracked per exam

Taking an exam does NOT advance time or change activity

Course: IN_PROGRESS $\rightarrow$ PASSED ($\geq 2/3$ passed & all 3 attempted) or FAILED ($\geq 2/3$ failed)

WIN / LOSE · STATE MODEL

GRADUATE (all): every course PASSED, `energy` > 0 , `happy` > 0

DROP OUT (any): a course FAILED · 5 total exam fails · `energy`=0 · `happy`=0

Re-checked after every action and every exam

Know which fields are stored vs derived; `passChance` etc. are for YOUR features

The prescribed interface (automarked)

Function	Parameters	Returns (no error)	Errors – return the FIRST that applies
authRegister	(studentId, password, nameFirst, nameLast)	{ studentId }	INVALID_STUDENT_ID · INVALID_FIRST_NAME · INVALID_LAST_NAME · INVALID_PASSWORD
authLogin	(studentId, password)	{ studentId }	INVALID_CREDENTIALS
simulationCreate	(studentId, universeData)	{ simulationId }	INVALID_STUDENT_ID · INVALID_UNIVERSE · MAX_SIMULATIONS (5 max)
simulationStatus	(studentId, simulationId)	{ simulationStatus, currentActivity, energy, happiness, timeElapsed }	INVALID_STUDENT_ID · INVALID_SIMULATION
clear	()	{ }	– none –

Validation - check the spec

Conventions & your designed functions

RETURN & VALIDATION CONVENTIONS

Errors: { error: 'ERROR_TYPE', message: '...' }

message is NOT marked — say anything useful

Two errors apply? Return the one listed FIRST in the table

You CAN assume correct datatypes are given

→ don't test for wrong types

You MUST validate everything else (formats, schema, ranges)

File = first word of the function name

YOUR DESIGNED FUNCTIONS

Implement the functions your group designed in Iter 0

Update your interface table to add error returns — same style as the prescribed table

Test them comprehensively + demo them to your tutor

Manually marked - your tutor will ask you to demo tests running

Tip: order common action errors INVALID_ACTION, INVALID_DURATION, SIMULATION_ENDED

Live demo

Red → Green: write the test before the code, then make it pass.

STEPS (run inside project-backend)

1. `npm test` → provided echo test passes
2. write `src/auth.test.js` – a happy path + an `INVALID_STUDENT_ID` case, with `clear()` in `beforeEach`. `npm test` → RED
3. implement minimal `authRegister` (ESM export) in `src/auth.js`. `npm test` → GREEN
4. `npm run coverage` → show statement %

- Tests are black-box — test through the interface
- `clear()` in `beforeEach` keeps tests independent
- `toStrictEqual` for objects; `expect.any(String)` for message
- 90% statement coverage = full coverage marks
- **Code without tests is assumed not to work**

Workflow

1

issue + AC

2

branch

3

write tests

4

implement

5

open MR

6

peer review

7

creator merges

THE RULES (spec 3.5)

- master is protected
- MR reviewed by someone who didn't write it; creator merges after approval
- **Never merge failing tests or untested features - both are penalised**
- Write some tests before the implementation

WORKING IN PARALLEL (spec 3.8)

- Be aware of the register→create→status dependency chain
- **git pull origin <branch> to share an in-progress branch**
- Pull master regularly; coordinate order in standups
- Each member: ≥ 1 MR on prescribed + ≥ 1 on a designed function

Where the marks are

Git, PM & Teamwork

35%

General Code Quality

20%

Automarking (tests + impl)

20%

Test Quality

15%

Interface Design

10%

THE AUTOMARK FORMULA

$$\text{Mark} = t \cdot i \cdot \min((c+10)/100, 1)^2$$

- t = your tests run against your code
- i = hidden course tests against your code
- c = statement coverage (+10, capped 100)

→ 90% coverage earns full coverage marks

A 'working' solution is not, by itself, a pass. Demo may also affect code, test and interface design marks (45%).

Before you submit

DRYRUN (on a CSE machine)

```
cd project-backend
1531 dryrun 1
cat ~cs1531/bin/iter1.test.cjs  # view tests
```

Checks return shapes + simple behaviour – not correctness

DEADLINE

12pm Monday, Week 5

Demonstrated in YOUR Week 5 lab · 35% of the project mark · no late submissions

THE PATH

Merge new starter (vite + ESM) → satisfy the prescribed interface → prove it with tests → check coverage → merge reviewed MRs

Read spec sections 3, 5, 6 and 7 in full this week. Keep master green - it IS your submission.